

Φ-253 Final Project:

Lunar surface satellite image enhancement for tracking appropriate landing zones using VAEs and Riemannian Geometry.

Contents:

1: Motivation

2: Dataset

3: Methodology

- *3.1 Finding the smoothest region*
- *3.2 The Variational Autoencoder (VAE)*
- *3.3 The use of Riemannian Geometry*
- *3.4 The mathematics Regarding Riemannian Geometry*
- *3.5 Parameter Selection and total Loss Function of the Complex Model*
- *3.6 Evaluation Metrics*

4: Results

- *Loss Function behaviour during training:*
- *Image Reconstruction Results*
- *Landing Zone Choice*

5: Conclusions

6: Future considerations

1. Motivation

Lunar ground missions are of great importance to the scientific community offering invaluable insights into lunar geology and providing data for mapping, similar to the Apollo missions. More recent missions, such as TRIDENT and MSOLO, utilize drills and various instruments to conduct biological experiments. These efforts are crucial for drawing conclusions about establishing a sustained human presence in lunar regions, a task of immense significance and complexity.

Getting scientific equipment like rovers or satellites with the necessary instruments out of orbit is a task of great complexity. However through the recent years many advancements have been made based on the data of previous launches, making reaching outer orbit much more reliable. Getting the equipment to land on the moon is no easy feat and there are far less data on how to do it in an efficient way without compromising the mission by damaging the equipment, or getting it stuck in a crater. Of course a lot of work is done prior to the landing phase in order to avoid such scenarios and find a reliable landing zone. Nevertheless, since there is an abundance of lunar surface images captured by space missions such as LROC and others, we could use these data in order to automate the process by training a neural network to be able to decide the most suitable landing zone. This project explores whether it is possible to use these images to train a neural network that can autonomously identify the most suitable landing zone, while also accounting for factors such as image noise from various interferences.

2. Dataset

The initial dataset that was collected for this model is comprised of 2500 high resolution images that were taken from im-ldi.com. This website contains several lunar surface reconnaissance missions that included the use of various satellites orbiting the moon. In our case the subset of the Apollo metric images were selected.

Not all of the images included in the subset were suitable for our intends and purposes for various factors such as lighting conditions, image formats etc. So we used filters such as a local sun location of 33-68 degrees.

More filtering had to be conducted as some images contained parts of the spacecraft as well, which is something that would hinder our goal to train on a suitable dataset.

After all this filtering we ended up with 1990 images that are suitable for training.

In the end due to hardware limitations in which we will mention in detail later we ended up training the model on only 200 images.

3. Methodology

The core of this project implements a two-stage approach in order to track the most optimal landing zone. It features Variational Autoencoders (VAEs), a powerful type of neural network suitable for learning latent representations and a custom method for quantifying lunar terrain smoothness.

3.1 Finding the smoothest region:

In this project we define smoothness based on the local texture and variance of the terrain. The `calculate_smoothness` function iterates over the final reconstructed lunar terrain images and calculates the variance of the pixel values within two windows.

- An 8x8 pixel window which assesses the terrain smoothness of the landing zone to ensure safe landing.
- A 32x32 pixel window which assesses whether the landing zone is isolated (on top of a hill, or inside a crater) to ensure that the equipment will be able to navigate to its target zone without getting stuck.

3.2 The Variational Autoencoder (VAE):

In general a VAE is a type of neural network that learns to compress input data, in this case lunar surface images, into a lower-dimensional latent space and then reconstructs the original data from that representation. In this project, the VAE is trained to denoise images by effectively learning to fill in missing information and by smoothing out imperfections. The latent space is a representation of a compressed yet meaningful summary of the terrain features.

To train the VAE, Gaussian noise was added to the input images. This is a common technique to make a model more efficient at training and to increase robustness. However based on the current context it also serves as a simulation of real-world noise that could occur from solar interference or the satellite's other instruments. In general a Gaussian noise value of 0.1 can serve as a simulation of the fore-mentioned but in our case we opted for a 0.3 value in order to test the network's capability to produce meaningful and utilizable results even with input data in worsened conditions.

The training process consists of two parts in the form of transfer learning

- A simpler Hyperspherical VAE is first trained to learn basic latent representation. Its encoder projects data onto the surface of a hypersphere, a method that can greatly improve the representation of complex and high-dimensional data.
- A more advanced Complex VAE is then build with a U-Net-like architecture and skip connections. It uses the weights of the pre-trained Hyperspherical VAE as a starting point. This speeds up the training process in a significant manner and improves the overall quality of the final reconstructions by implementing the initial knowledge learned by the simpler model.

3.3 The use of Riemannian Geometry

In this project the latent space is not a flat Euclidean space but a curved Riemannian manifold, a hypersphere to be more precise. This can prove highly beneficial for the task at hand for a variety of reasons:

- **Capturing Complex Relationships:**

Lunar images contain features like craters, shadows and slopes. A flat Euclidean latent space would not be optimal for this task due to its struggle to represent the non-linear relationships that these features embody. Through the use of Riemannian Geometry the model is better equipped to handle mapping this complex, high-dimensional data onto a curved surface while simultaneously preserving the structure of the lunar terrain.

- **Structured and Continuous Latent Space:**

The geometric constraints imposed by the Riemannian Geometry makes the model learn a better structured and continuous latent space. The similar terrain features are more likely to be assigned in the same groups which allows for smoother and realistic interpolations between different types of surfaces. This makes the model better at generalizing to new unseen images.

3.4 The mathematics regarding Riemannian Geometry:

All the above are attributed to Riemannian Geometry's intrinsic structure. The core of a hyperspherical VAE's behaviour is based on it implementing a latent space on a curved manifold. Instead of the standard Gaussian Distribution the model uses the Fisher-von Mises distribution as its prior for the latent space. This distribution is the natural probability distribution on a hypersphere.

The HypersphereSampling layer uses a re-parametrization trick to sample from this distribution. A point z is first sampled from a standard Gaussian Distribution and then gets projected to the surface of the hypersphere with an L2 normalisation.

$$z = L2_{Normalization}(\mu + e^{\kappa} \odot \epsilon)$$

Where μ is the mean, κ is the concentration parameter learned by the model and ϵ is a random variable that is sampled from a standard Gaussian Distribution.

The model's loss function includes a specialised KL divergence term that measures the difference between the learned latent distribution and the Fisher-von Mises prior. The exact formula that is used in the `hypersphere_kl_divergence` function is a simplified yet effective approximation of the term listed below:

$$L_{KL} = \frac{1}{N} \sum_{i=1}^n (\kappa_i - \log(\kappa_i))$$

This term penalizes deviations from the ideal form of a spherical distribution, which encourages the model to learn a well structured and continuous latent space in respect to the specific geometry of the hypersphere.

3.5 Parameter Selection and total Loss Function of the Complex Model:

Regarding the parameters of the networks we opted for a batch size of 8 in order to balance training performance and memory usage.

The learning rate was adjusted to 0.0005 in the Adam optimizer but the ReduceLROnPlateau scheduler adjusted it dynamically during training in order to prevent overshooting the minimum loss.

In the complex VAE we opted for a custom loss function that is comprised by the following:

- Reconstruction Loss (MSE, weight=1.0): This is the primary component of the loss function that measures the pixel-by-pixel difference between the reconstructed image and the original image (before the application of Gaussian Noise). The loss is calculated by the Mean Squared Error as follows:

$$MSE = \frac{1}{n} \sum_i^n (Y_i - \hat{Y}_i)^2$$

- Perceptual Loss (weight=0.1): It is calculated by passing both the original and reconstructed images through a pre-trained **VGG16** network. The loss is then the Mean Squared Error between the feature maps from an early layer of the **VGG16** network.. This choice ensures that the reconstructed image is not only pixel perfect, but also similar structurally and visually to the original image. These factors play an important role in capturing the “feel” of the terrain.
- KL Divergence Loss (weight=0.1): This term ensures that the latent space representation is well structured and continuous which on its own prevents the VAE from simply memorizing the input data and thus avoiding overfitting.

3.6 Evaluation metrics:

The chosen evaluation metric for the project’s results is the Structural Similarity Index (SSIM). It is a perceptual metric that evaluates image quality by comparing the contrast, luminance and other structural information between a distorted image and a reference. Its values range from -1.0 to 1.0 where -1 is interpreted as perfect anti-similarity and 1 as a perfect similarity between the two images.

Also in order to asses from a different perspective the improvement that this approach brings to the table, we decided to include the PSNR metric. PSNR stands for Peak Signal to Noise Ratio and it measures the pixel quality of an image, a high PSNR value (measured in dB) indicates high quality images and better noise reduction. We are going to compare the PSNR value of the noisy image in respect to the original one with the PSNR value of the reconstructed image in respect to the original one. After that by subtracting the two values from one another we can see the improvement the network brought to the table by using this formula:

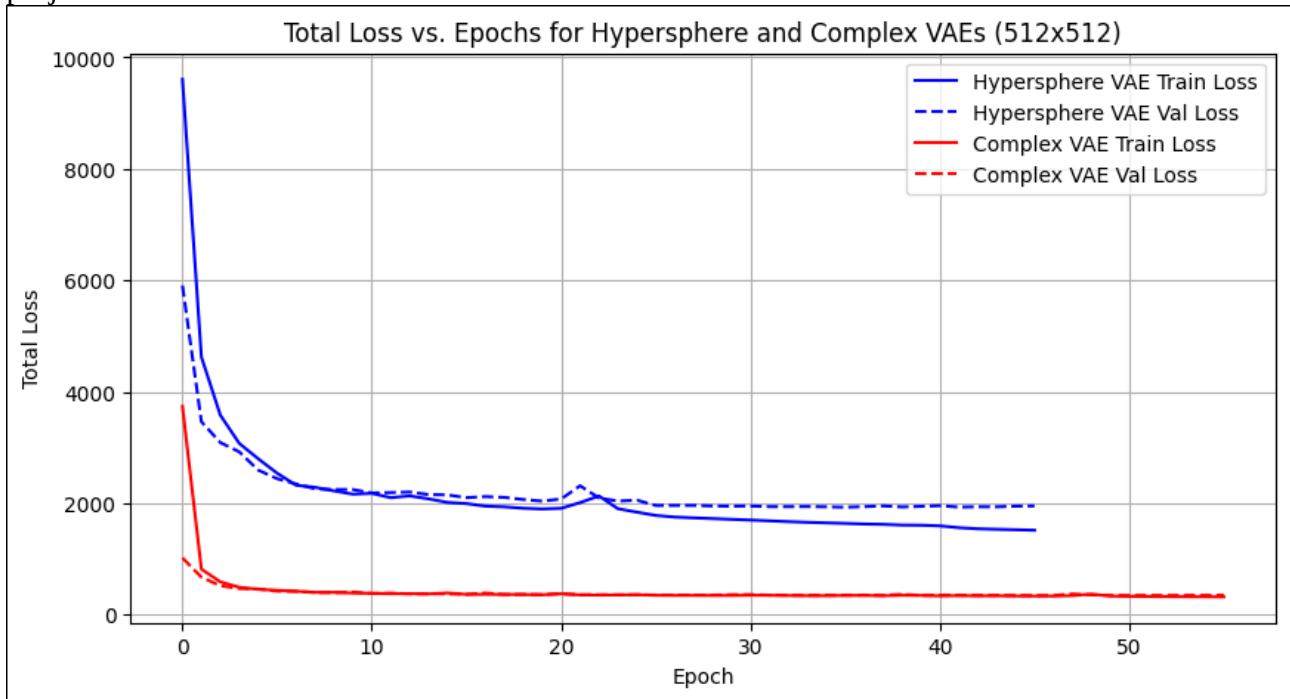
$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX^2}{MSE} \right) dB$$

(Where MAX is the maximum possible value of a pixel 255)

4. Results

Loss Function behaviour during training:

The graph below indicates the behaviour of the loss function for each of the VAEs used in the project.



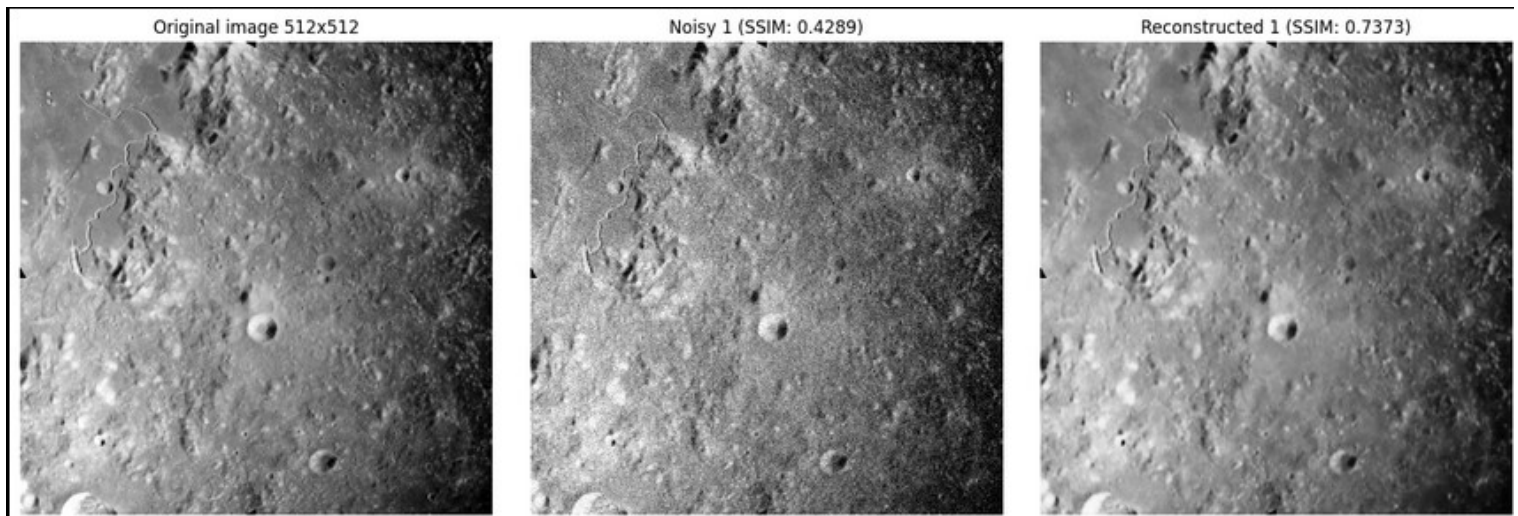
The hyperspherical VAE ended up with a loss equal to 1450 in the training set and 1918 in the validation set.

The complex VAE ended up with a loss equal to 323 in the training set and 339 in the validation set.

These loss values are the first indication of the model's success. It generalizes well and it is not prone to overfitting while at the same time it achieves satisfactory low values of reconstruction loss.

Image Reconstruction Results:

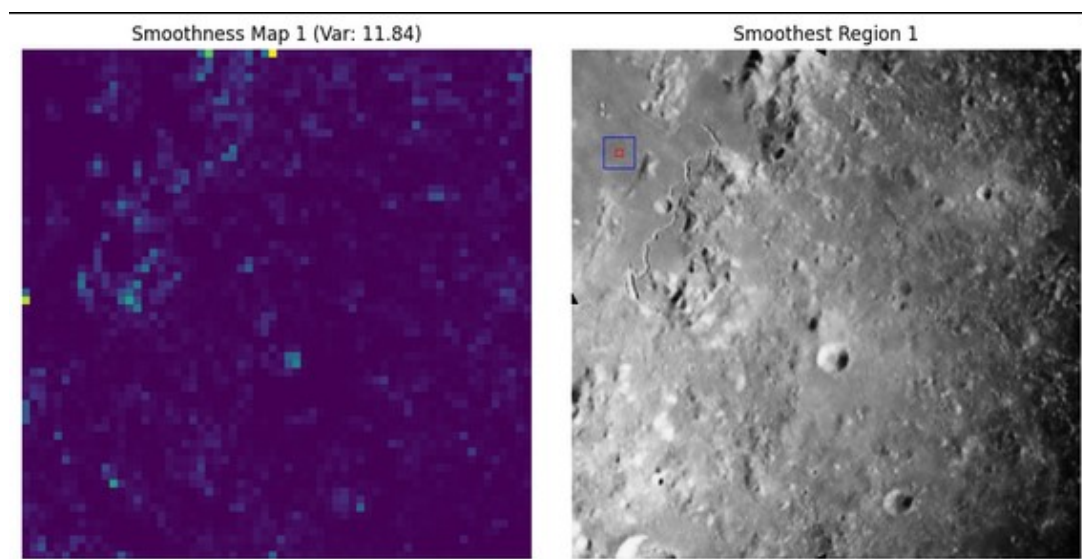
The final output of the code is a set of 5 images. The first three are the original image, the image after the noise addition and the reconstructed image as seen below:



When the original image is subjected to Gaussian Noise its SSIM value drops significantly. The average yielded SSIM value for the noisy images was 0.368. However when the training process is completed the complex VAE attempts to reconstruct the original image with great success reaching an average value of SSIM equal to 0.7051. Also the average PSNR value of the noisy images was 20.05dB, while their reconstructed counterpart had an average PSNR value of 28.11dB. This means that the noise has been reduced by 85%.

Landing Zone Choice:

Below are listed last two images of the code's output. The variance based smoothness map and the most optimal landing zone that the model predicted.



As clearly visible the landing zone selection works the way it is supposed to and tracks down an LZ that is smooth enough for landing and makes sure that the immediate surrounding environment will not pose any limitation regarding a rover's movement.

5. Conclusions

This project was developed and ran in a local setup. Given the compromises that we had to make due to hardware limitations such as the reduced Batch Size, the small Dataset and the relatively small resolution due to the rescaling of the images and the small amount of VAE layers, I would say that the model's results exceeded initial expectations.

The yielded PSNR average values indicate that the model is performing well when it comes to reducing noise. The average SSIM value of the reconstructed images in comparison to the original ones was 0.7051. While this value is not a good match to the task's standards, I believe that given the pipeline's functionality a larger dataset and better hardware would enhance this value further and result in even more accurate outputs.

6. Future Considerations

It would be interesting to explore whether the model can function with real time processing. This would provide the model with even more accurate data as the spacecraft approaches the lunar surface during landing. Also if this implementation were to be accurate enough, we could automate trajectory corrections during the approach and thus assisting the mission control's center.